



Abstract Factory Pattern

Human Computer Interaction Research
University of Nevada, Reno



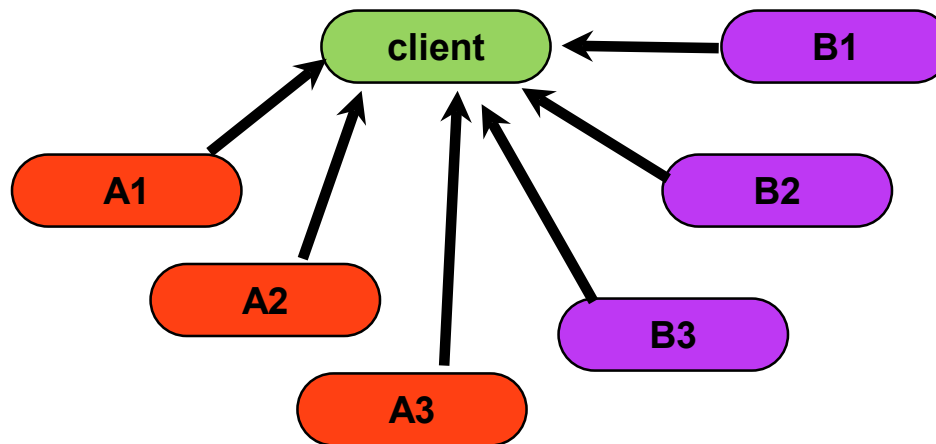
Pattern Categories

- Structural Patterns
 - » decorator
- Behavioral Patterns
 - » observer
 - » strategy
- Creational Patterns
 - » factory method
 - » abstract factory

Problem

“Too many dependencies to concrete classes makes your software difficult to maintain and modify”

example



client depends on six classes

Dependency Inversion principle

“Depend upon Abstractions, do
not depend upon concrete
classes”



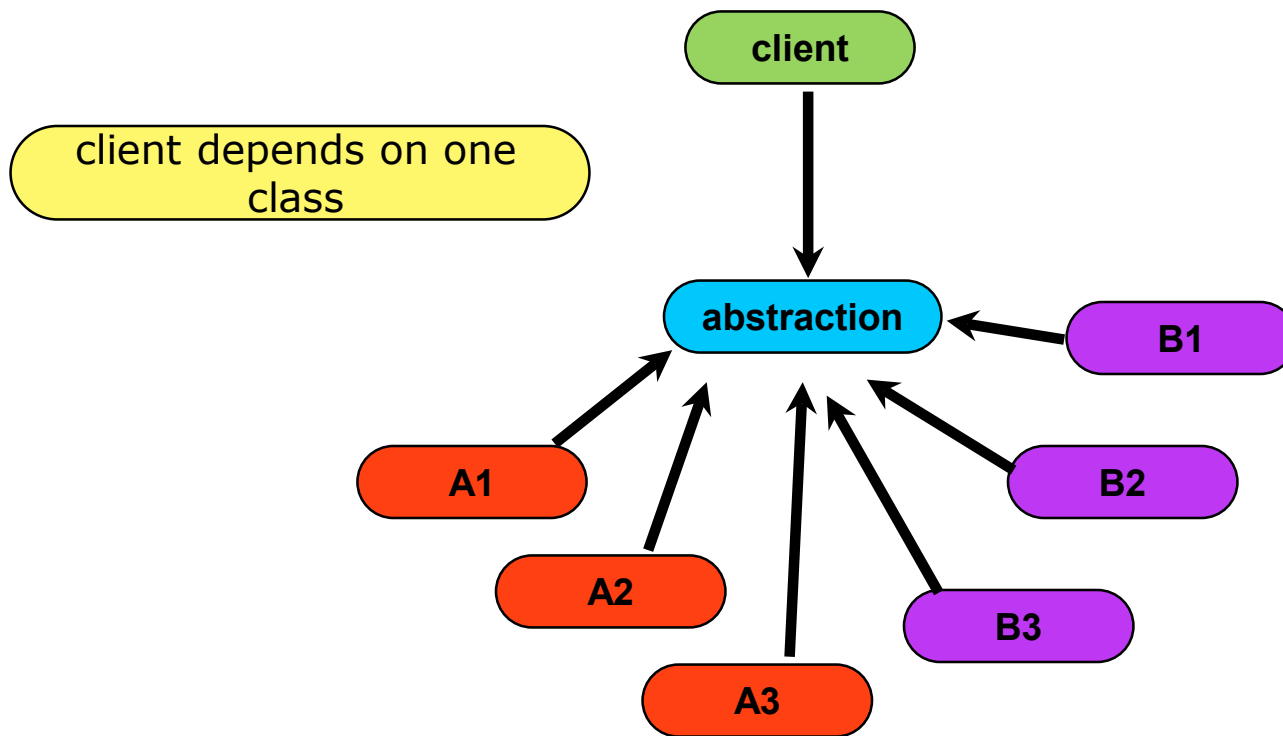
DEPENDENCY INVERSION

Would you solder a lamp directly to the electrical wiring in a wall?

Guidelines

- ☐ No variable should have a reference to a concrete class
- ☐ No class should derive from a concrete class
- ☐ No method should override an implemented method of any of its base classes.

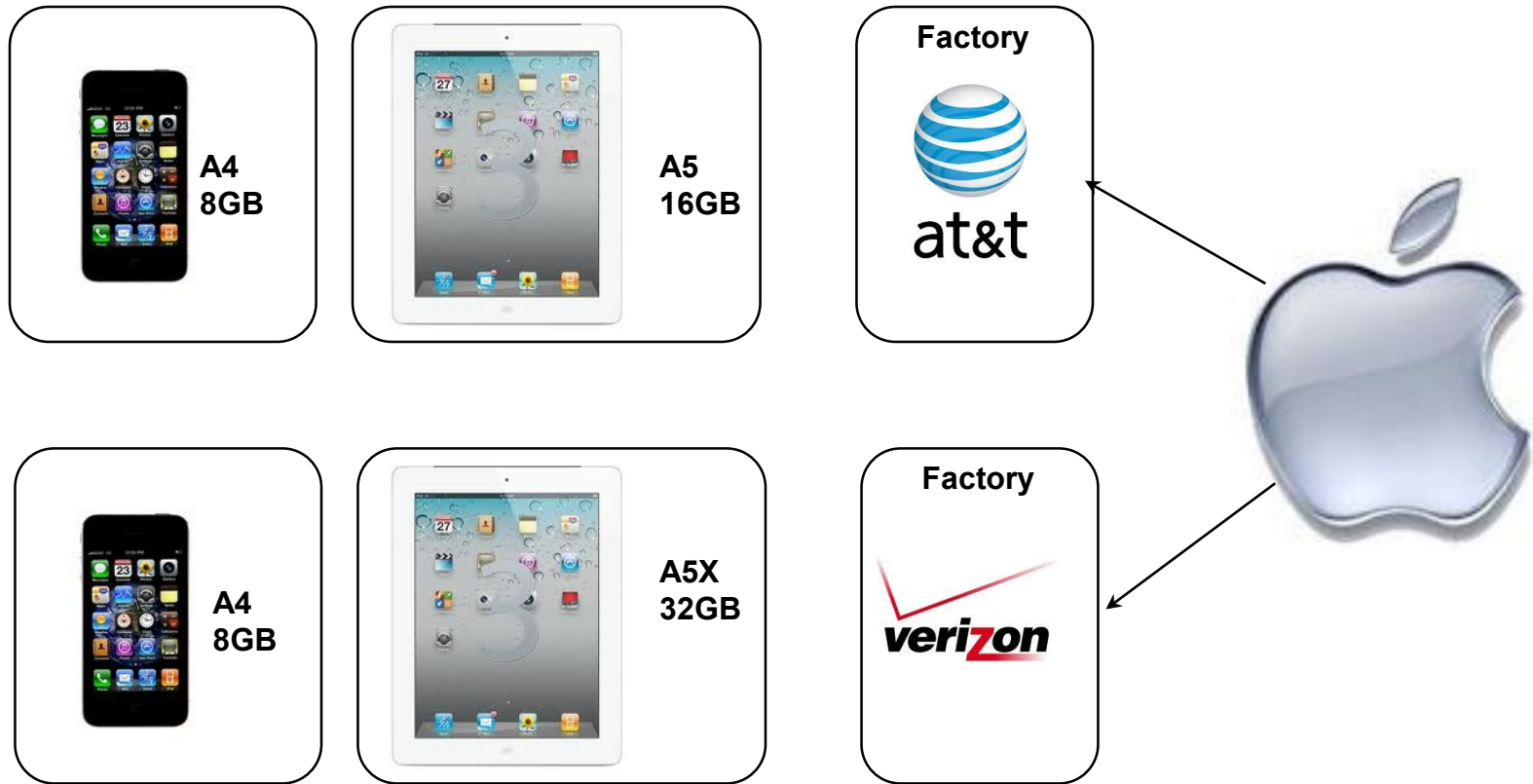
Factory Method



Example

iPad	iPhone	
		
		

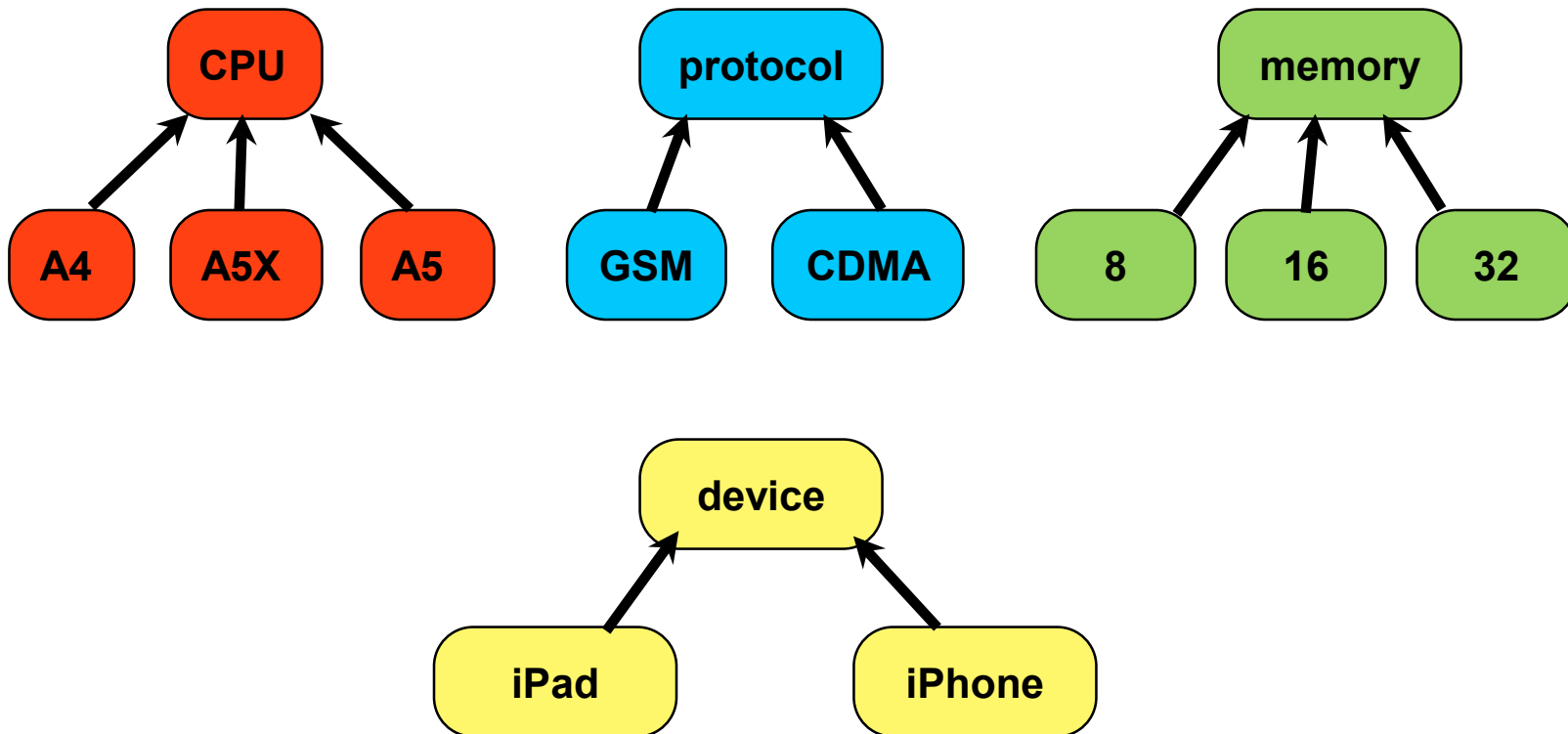
Instead of creating this



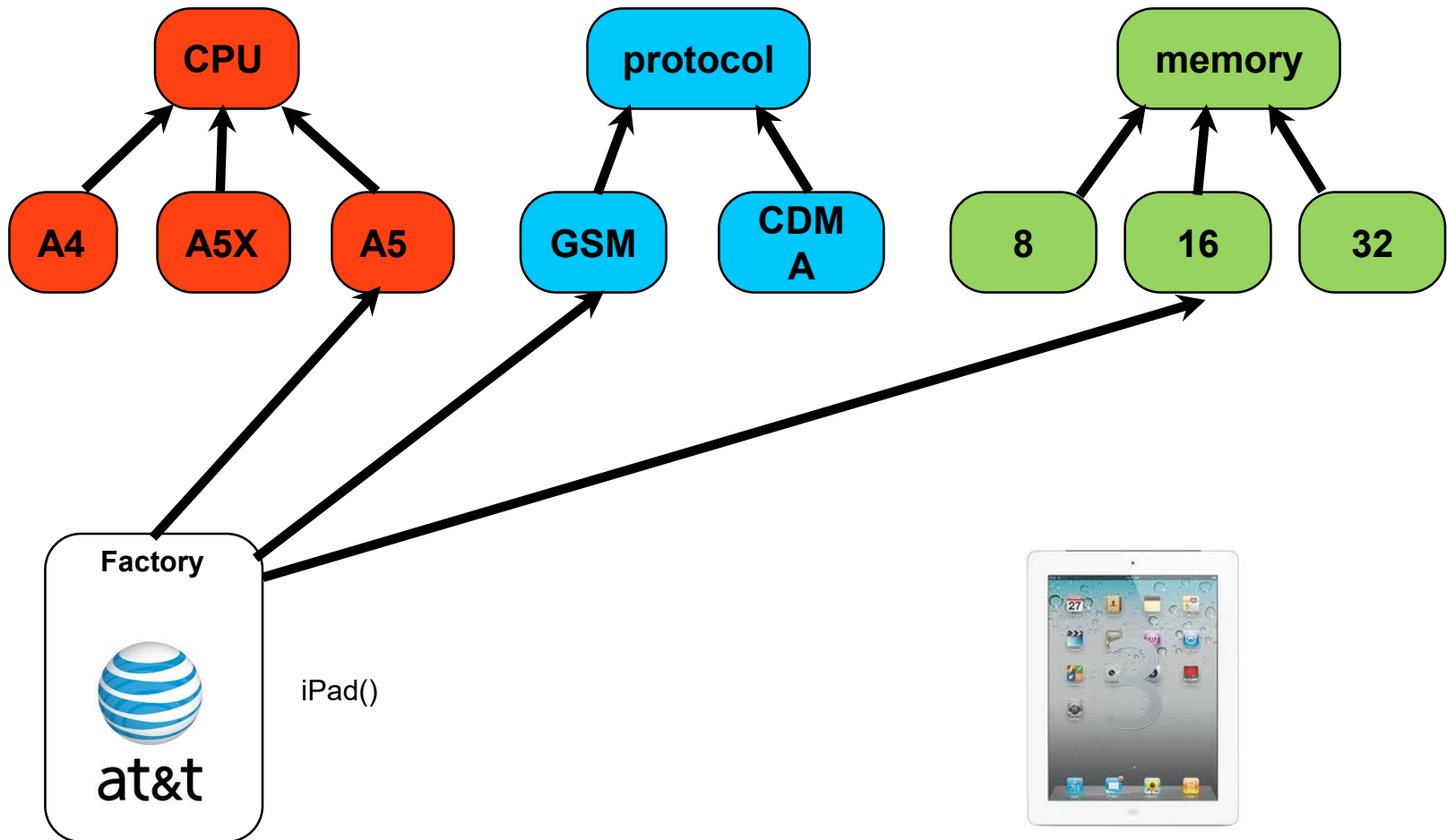
Look at “ingredients”

		AT&T	Verizon
iPhone	cpu	A4	A4
	protocol	GSM	CDMA
	memory	8	16
iPad	cpu	A5	A5X
	protocol	GSM	CDMA
	memory	16	32

Create Abstractions



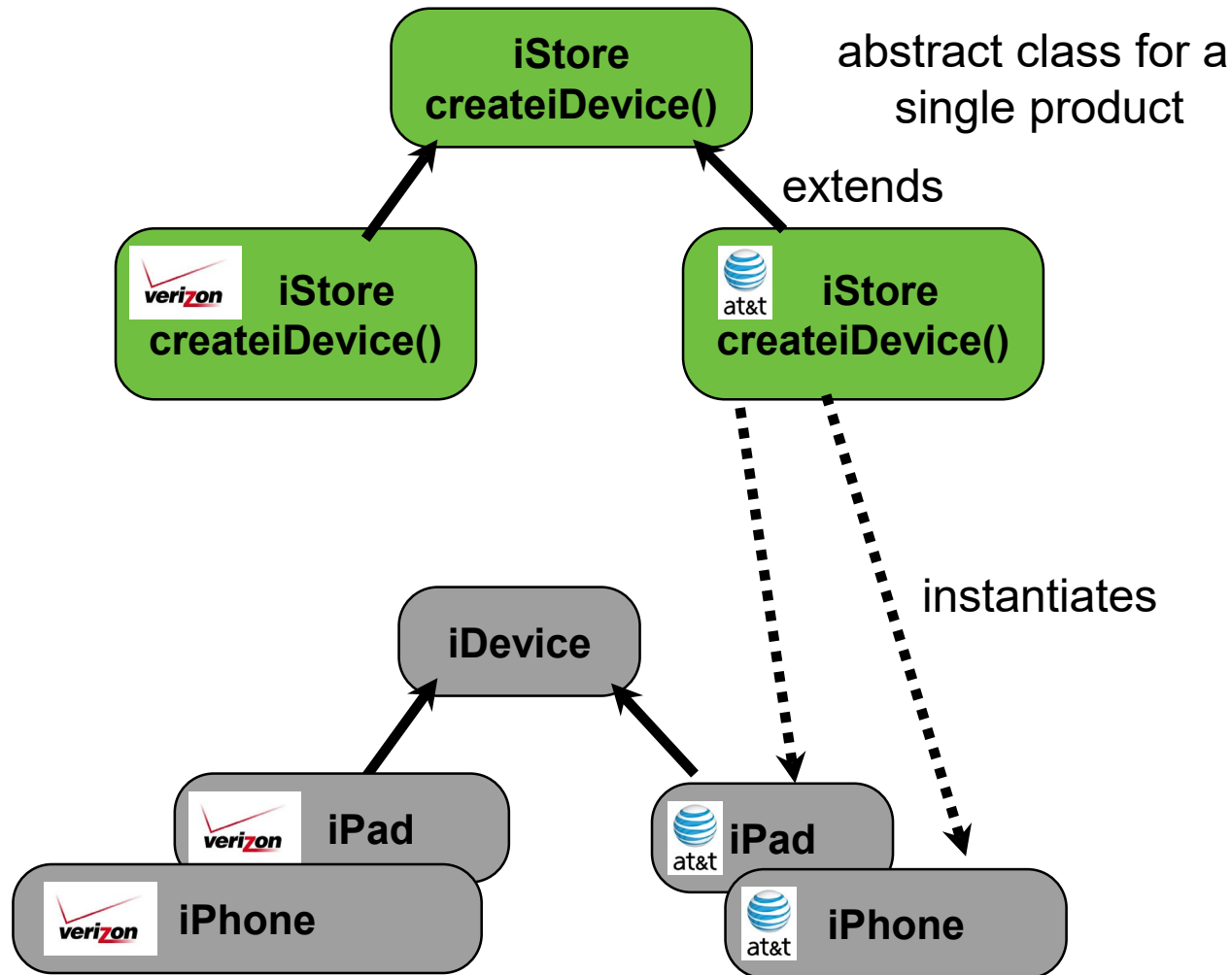
Create Ingredient Factory



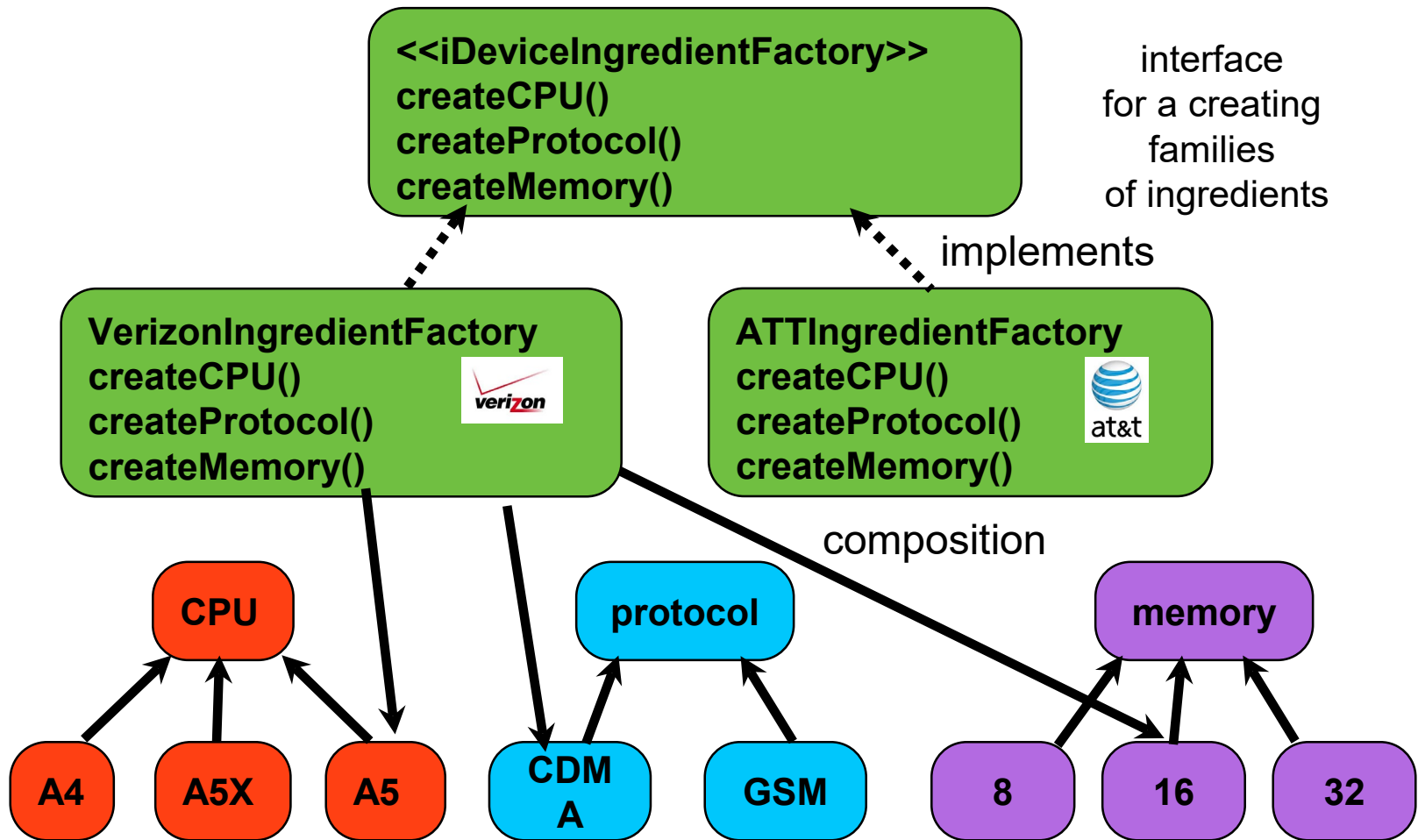
Abstract Factory Pattern

The Abstract Factory Pattern provides an interface for creating families of related or dependent objects without specifying their concrete classes.

Difference: factory method



Difference: abstract factory



Exercise II



projector

desk

Group 1

Group 2

Group 3

Group 4

Group 5

Group 6

Group 7

Group 8

Group 9

Group 10

Group 11

Group 12

Group 13

Group 14

Group 15

Group 16

Group 17

Group 18

Group 19

Group 20

Group 21

Group 22

Group 23

Group 24

Group 25

Group 26

Group 27

Group 28

Group 29

pillar

skateboards

door